

Facultad de Ingeniería

Ingeniería Civil Informática

**PLATAFORMA DE PREDICCIÓN DE
DEMANDA Y
REABASTECIMIENTO AUTOMÁTICO
PARA
DISTRIBUIDORAS CHILENAS
MEDIANTE
SELECCIÓN AUTOMÁTICA DE
MODELOS (AMS)**

Autores:

Matias Muñoz Parraguirre

Ignacio Duarte Miranda

Cristóbal Flores Villegas

Profesor:

Paulo Luis Francisco Quinsacara Jofré

Santiago, Chile

May 11, 2026

Declaración de Originalidad

Declaramos que este trabajo de tesis ha sido desarrollado de forma íntegra y original por los autores indicados en la portada. Todo el material bibliográfico utilizado ha sido debidamente referenciado según las normas APA 7.^a edición. No se ha hecho uso de inteligencia artificial generativa para producir contenidos sustantivos del documento sin su declaración explícita, conforme a las directrices de la Facultad de Ingeniería de la Universidad Andrés Bello.

Integrante 1

Integrante 2

Cristóbal Flores Villegas

Contents

Declaración de Originalidad	1
Resumen	7
Abstract	8
1 Introducción	9
1.1 Contexto	9
1.2 Importancia del Proyecto	9
1.3 Contribución del Proyecto	9
1.4 Objetivos	10
1.4.1 Objetivo General	10
1.4.2 Objetivos Específicos	10
1.5 Limitaciones del Proyecto	11
2 Marco Teórico	12
2.1 Metodologías de Trabajo	12
2.1.1 Metodología Scrum	12
2.1.2 ¿Cómo funciona Scrum?	12
2.1.3 Roles de Scrum	12
2.1.4 Fases de una Metodología Scrum	13
2.2 Proceso CRISP-DM	13
2.3 Modelos de Forecasting	13
2.3.1 ARIMA	13
2.3.2 Prophet	13
2.3.3 XGBoost	13
2.3.4 LSTM	14
2.4 Selección Automática de Modelos (AMS)	14
2.5 Gestión de Inventario	14
2.5.1 Modelo EOQ	14
2.5.2 Política (s, S)	14

2.6	Dataset: Store Sales — Corporación Favorita	14
3	Identificación de Requisitos	16
3.1	Partes Interesadas (Stakeholders)	16
3.2	Usuarios	16
3.3	Roles	16
3.4	Requisitos del Sistema	16
3.5	Planificación Sprint	19
4	Product Backlog y Sprint Backlog	20
4.1	Product Backlog	20
4.2	Sprint Backlog — Sprint 1	21
5	Arquitectura del Sistema	22
5.1	Visión General	22
5.2	Diagrama de Componentes (Vista de Desarrollo)	22
5.3	Diagrama de Base de Datos (Vista Lógica)	23
5.4	Diagrama de Secuencia — Predicción de Demanda	23
5.5	Diagrama de Despliegue (Vista Física)	24
5.6	Módulo de Ingesta Inteligente	24
5.6.1	Flujo de Ingesta	24
5.6.2	Prompt de Extracción	25
6	Desarrollo	26
6.1	Pipeline ETL	26
6.2	Módulo de Forecasting	26
6.3	Módulo de Inventario	26
6.4	API REST	27
6.5	Dashboard Web	27
7	Pruebas y Resultados	28
7.1	Pruebas Unitarias	28
7.2	Resultados del Módulo AMS	29
8	Retrospectiva del Sprint	30
8.1	Comprobación de Sprint 1	30
8.2	Retrospectiva del Sprint 1	30
8.3	Gráfico Burn-down	31
9	Conclusiones	32
9.1	Conclusiones respecto a los Objetivos	32

9.1.1	Objetivos Específicos	32
9.1.2	Objetivo General	33
9.2	Conclusión respecto a la Metodología Scrum	33
9.3	Proyecciones	33
Referencias		34

List of Figures

2.1	Metodología Scrum	12
5.1	Arquitectura de componentes de SmartSupply	22
5.2	Diagrama de Base de Datos de SmartSupply	23
5.3	Diagrama de Secuencia — Predicción de demanda via AMS	24
5.4	Infraestructura de despliegue de SmartSupply (Cloudflare Pages + Render + Supabase)	24
8.1	Burn-down del Sprint 1	31

List of Tables

2.1	Roles Scrum — SmartSupply	12
2.2	Características del Dataset Store Sales — Corporación Favorita	15
3.1	Stakeholders de SmartSupply	16
3.2	Requisito RF-01: Ejecutar Predicción de Demanda por SKU	17
3.3	Requisito RF-02: Generar Alerta de Reabastecimiento	17
3.4	Requisito RF-03: Crear Orden de Compra Automática	17
3.5	Requisito RF-04: Visualizar Dashboard de Métricas	18
3.6	Requisito RF-05: Registrar Negocio en el Sistema	18
3.7	Requisito RF-06: Ingestar Datos de Ventas mediante IA	18
3.8	Planificación de Sprints	19
4.1	Product Backlog — SmartSupply	20
4.2	Sprint Backlog — Sprint 1: Setup y ETL Base	21
5.1	Arquitectura en Cuatro Capas	22
6.1	Endpoints de la API REST	27
7.1	Prueba Unitaria PU-01: Predicción de Demanda	28
7.2	Prueba Unitaria PU-02: Alerta de Reabastecimiento	28
8.1	Recuento de Tareas — Sprint 1	30

Resumen

La gestión eficiente del inventario es uno de los desafíos más críticos para las distribuidoras chilenas, donde la variabilidad estacional de la demanda y la diversidad de productos dificultan la aplicación de un único modelo de predicción. El presente proyecto propone **SmartSupply**, una plataforma de predicción de demanda y reabastecimiento automático que implementa un mecanismo de Selección Automática de Modelos (AMS) por SKU, evaluando cuatro algoritmos —ARIMA, Prophet, XGBoost y LSTM— y seleccionando aquel con menor error de predicción (MAPE) para cada familia de productos.

La plataforma integra un pipeline ETL sobre el dataset público *Store Sales — Corporación Favorita* (Kaggle, 2017), una API REST construida con FastAPI, un módulo de gestión de inventario basado en las políticas EOQ y (s, S) , y un dashboard web desarrollado en React. Adicionalmente, el sistema incorpora un **módulo de ingesta inteligente** que utiliza el modelo de lenguaje Claude (Anthropic) con capacidades de visión para extraer, interpretar y normalizar datos de ventas desde cualquier formato — imágenes de cuadernos, planillas Excel o PDFs — permitiendo que distribuidoras chilenas sin sistemas digitalizados incorporen sus datos al pipeline de forecasting. La arquitectura es **multi-tenant**: cada negocio registrado opera de forma aislada sobre la misma base de datos mediante un identificador único (`business_id`). La metodología de desarrollo sigue el framework ágil Scrum y el proceso CRISP-DM para el tratamiento de los datos.

Palabras clave: predicción de demanda, selección automática de modelos, MAPE, inventario, EOQ, política (s, S) , ARIMA, Prophet, XGBoost, LSTM, FastAPI, React, ingesta IA, multi-tenancy, Claude, visión por computador.

Abstract

Efficient inventory management is one of the most critical challenges for Chilean distributors, where seasonal demand variability and product diversity make it difficult to apply a single forecasting model. This project proposes **SmartSupply**, a demand forecasting and automatic restocking platform that implements an Automatic Model Selection (AMS) mechanism per SKU, evaluating four algorithms —ARIMA, Prophet, XGBoost, and LSTM— and selecting the one with the lowest forecasting error (MAPE) for each product family.

The platform integrates an ETL pipeline over the public dataset *Store Sales — Corporación Favorita* (Kaggle, 2017), a REST API built with FastAPI, an inventory management module based on EOQ and (s, S) policies, and a web dashboard developed in React. Additionally, the system incorporates an **intelligent ingestion module** that uses the Claude language model (Anthropic) with vision capabilities to extract, interpret, and normalize sales data from any format — handwritten notebook photos, Excel spreadsheets, or PDFs — enabling Chilean distributors without digital systems to incorporate their data into the forecasting pipeline. The architecture is **multi-tenant**: each registered business operates in isolation on the same database through a unique identifier (`business_id`). The development methodology follows the Scrum agile framework and the CRISP-DM process for data treatment.

Keywords: demand forecasting, automatic model selection, MAPE, inventory, EOQ, (s, S) policy, ARIMA, Prophet, XGBoost, LSTM, FastAPI, React, AI ingestion, multi-tenancy, Claude, computer vision.

1. Introducción

1.1. Contexto

Las distribuidoras chilenas operan en un entorno de alta competitividad donde el control del inventario determina directamente la rentabilidad del negocio. Un quiebre de stock implica ventas perdidas y deterioro de la relación con el cliente, mientras que un exceso de inventario inmoviliza capital de trabajo. Ambos escenarios son consecuencia de predicciones de demanda imprecisas o metodologías de reabastecimiento estáticas.

La proliferación de algoritmos de *machine learning* y series de tiempo ofrece hoy múltiples alternativas para el pronóstico de ventas, pero la elección del modelo óptimo varía según las características de cada producto (estacionalidad, tendencia, presencia de ceros) y el horizonte de predicción requerido.

1.2. Importancia del Proyecto

La selección inadecuada del modelo de forecasting es una de las causas más frecuentes de error en los sistemas de reabastecimiento. Un modelo que funciona bien para productos de alta rotación puede ser completamente inadecuado para artículos con demanda esporádica. Automatizar esta selección por SKU permite:

- Reducir el MAPE global de predicción respecto a un modelo único.
- Disminuir el capital inmovilizado en inventario.
- Generar órdenes de compra oportunas y calibradas a la demanda real.
- Entregar a los gestores una herramienta visual de monitoreo en tiempo real.

1.3. Contribución del Proyecto

SmartSupply aporta:

1. Un **módulo AMS** que selecciona automáticamente el mejor modelo de forecasting por SKU mediante validación walk-forward (70 / 15 / 15).

2. Un **módulo de inventario** que aplica EOQ y política (s, S) sobre los pronósticos generados por AMS.
3. Una **API REST** (FastAPI) que expone los resultados a cualquier cliente, documentada automáticamente via OpenAPI/Swagger.
4. Un **dashboard web** (React + Recharts) para visualización de pronósticos, alertas y órdenes de compra.
5. Un **pipeline ETL** reproducible sobre un dataset real de ventas minoristas.
6. Un **módulo de ingesta IA** que utiliza Claude (Anthropic) para extraer y normalizar datos de ventas desde imágenes, planillas Excel y PDFs, permitiendo incorporar al sistema a distribuidoras sin datos estructurados.
7. Una **arquitectura multi-tenant** que aísla los datos de cada negocio mediante `business_id` y Row Level Security (RLS) en PostgreSQL, habilitando el despliegue como plataforma SaaS.

1.4. Objetivos

1.4.1. Objetivo General

Desarrollar una plataforma de predicción de demanda y reabastecimiento automático para distribuidoras chilenas que, mediante la selección automática del mejor modelo de forecasting por SKU, reduzca el error de predicción y el capital inmovilizado respecto al uso de un modelo único para todos los productos.

1.4.2. Objetivos Específicos

1. Implementar y evaluar cuatro modelos de predicción de series de tiempo (ARIMA, Prophet, XGBoost, LSTM) sobre el dataset Store Sales — Corporación Favorita.
2. Desarrollar el mecanismo AMS que selecciona el modelo de menor MAPE por SKU mediante validación cruzada temporal (walk-forward).
3. Implementar las políticas de gestión de inventario EOQ y (s, S) alimentadas por los pronósticos del AMS.
4. Construir una API REST con FastAPI que exponga los módulos de forecasting e inventario a clientes externos.
5. Desarrollar un dashboard web interactivo en React para la visualización de pronósticos, métricas de inventario y generación de órdenes de compra.

6. Diseñar e implementar un pipeline ETL que cargue, limpie y normalice el dataset de entrenamiento en una base de datos PostgreSQL.
7. Desarrollar un módulo de ingesta inteligente basado en IA (Claude, Anthropic) que permita a distribuidoras chilenas incorporar sus datos históricos de ventas desde cualquier formato digital (imágenes, Excel, PDF), eliminando la barrera de entrada para negocios sin sistemas ERP.

1.5. Limitaciones del Proyecto

- El sistema se valida sobre el dataset Store Sales — Corporación Favorita (Ecuador), que actúa como negocio de demostración (**business_id=1**). El módulo de ingesta IA permite incorporar datos de distribuidoras chilenas reales en producción.
- El AMS evalúa únicamente cuatro modelos de forecasting; existen alternativas (SARIMA, N-BEATS, etc.) no contempladas en este alcance.
- El módulo de inventario no implementa restricciones de capacidad de bodega ni descuentos por volumen.
- La calidad de la extracción del módulo de ingesta IA depende de la legibilidad del documento fuente; imágenes de muy baja resolución pueden producir resultados incompletos.
- La integración con sistemas ERP o WMS externos queda fuera del alcance de esta tesis.

2. Marco Teórico

2.1. Metodologías de Trabajo

2.1.1. Metodología Scrum

Scrum es un framework ágil para el desarrollo iterativo e incremental de productos complejos (Schwaber & Sutherland, 2020). Se organiza en ciclos de trabajo llamados *Sprints* de duración fija (en este proyecto, dos semanas), al final de los cuales se entrega un incremento potencialmente funcional del sistema.

2.1.2. ¿Cómo funciona Scrum?

El proceso Scrum comienza con la definición del *Product Backlog*, lista priorizada de todas las funcionalidades requeridas. En cada *Sprint Planning* el equipo selecciona los ítems a desarrollar en el siguiente Sprint, generando el *Sprint Backlog*. Diariamente se realiza una reunión breve (*Daily Scrum*) para sincronizar el avance. Al final del Sprint se realiza la *Sprint Review* (demo) y la *Sprint Retrospective* (mejora de proceso).

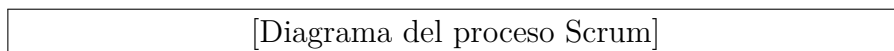


Figure 2.1: Metodología Scrum

2.1.3. Roles de Scrum

Table 2.1: Roles Scrum — SmartSupply

Rol	Responsabilidad	Integrante
Product Owner	Priorizar Product Backlog, validar entregables	Paulo Quinsacara Jofré
Scrum Master	Facilitar ceremonias, remover impedimentos	Integrante 1
Dev Team	Diseño, implementación, pruebas	Los 3 integrantes

2.1.4. Fases de una Metodología Scrum

Las fases principales son: Planificación del Sprint, Ejecución del Sprint (con Daily Scrums), Sprint Review y Sprint Retrospective. Este ciclo se repite durante los 6 Sprints planificados para SmartSupply.

2.2. Proceso CRISP-DM

CRISP-DM (*Cross Industry Standard Process for Data Mining*) es un proceso estándar para proyectos de ciencia de datos que consta de seis fases (Shearer, 2000):

1. **Business Understanding:** definir el problema de negocio.
2. **Data Understanding:** exploración y calidad de los datos.
3. **Data Preparation:** limpieza, transformación e integración.
4. **Modeling:** selección y entrenamiento de modelos.
5. **Evaluation:** validación contra métricas de negocio.
6. **Deployment:** puesta en producción del modelo.

2.3. Modelos de Forecasting

2.3.1. ARIMA

ARIMA (*Autoregressive Integrated Moving Average*) es un modelo estadístico clásico para series de tiempo estacionarias (Box & Jenkins, 1976). Se define por tres parámetros (p, d, q) : orden autorregresivo, grado de diferenciación y orden de media móvil.

2.3.2. Prophet

Prophet es un modelo de forecasting desarrollado por Meta (Taylor & Letham, 2018), diseñado para manejar estacionalidades múltiples, efectos de feriados y valores atípicos. Descompone la serie en tendencia, estacionalidad y calendario.

2.3.3. XGBoost

XGBoost (*Extreme Gradient Boosting*) es un algoritmo de ensamble basado en árboles de decisión (Chen & Guestrin, 2016). Para series de tiempo se aplica mediante ingeniería de características: lags, medias móviles, variables de calendario y features exógenas (precio del petróleo, feriados).

2.3.4. LSTM

Las redes LSTM (*Long Short-Term Memory*) son una variante de las redes neuronales recurrentes que resuelven el problema del gradiente desvanecido mediante compuertas de entrada, olvido y salida (Hochreiter & Schmidhuber, 1997). Son especialmente efectivas para capturar dependencias de largo plazo en series temporales.

2.4. Selección Automática de Modelos (AMS)

El AMS implementa validación *walk-forward*: el dataset se divide en 70 % entrenamiento, 15 % validación y 15 % prueba. Para cada SKU, todos los modelos se entrenan y evalúan sobre la ventana de validación; el de menor MAPE se designa como modelo activo para ese SKU. La fórmula del MAPE es:

$$\text{MAPE} = \frac{100\%}{n} \sum_{t=1}^n \left| \frac{y_t - \hat{y}_t}{y_t} \right|$$

2.5. Gestión de Inventario

2.5.1. Modelo EOQ

El *Economic Order Quantity* (EOQ) determina la cantidad óptima de pedido que minimiza los costos totales de inventario (Harris, 1913):

$$Q^* = \sqrt{\frac{2DK}{h}}$$

donde D es la demanda anual, K el costo fijo de pedido y h el costo de mantención por unidad por año.

2.5.2. Política (s, S)

La política de inventario (s, S) establece dos parámetros: el punto de reorden s (nivel mínimo que dispara un pedido) y el nivel objetivo S (nivel al que se sube el inventario tras el pedido). Cuando el inventario cae por debajo de s , se genera una orden por $Q = S - \text{inventario actual}$ unidades.

2.6. Dataset: Store Sales — Corporación Favorita

El dataset público Store Sales — Corporación Favorita (Kaggle, 2017) contiene datos reales de ventas de 54 tiendas en Ecuador durante el período 2013–2017. Sus características principales se resumen en la Tabla 2.2.

Table 2.2: Características del Dataset Store Sales — Corporación Favorita

Característica	Valor
Período	2013-01-01 a 2017-08-15
Filas (train)	3.000.888
Tiendas	54
Familias (SKUs)	33
Columnas	id, date, store_nbr, family, sales, onpromotion
Demanda cero	~31,3 % de los registros
Archivos extra	stores.csv, oil.csv, holidays_events.csv, transactions.csv

3. Identificación de Requisitos

3.1. Partes Interesadas (Stakeholders)

Table 3.1: Stakeholders de SmartSupply

Stakeholder	Interés	Influencia
Jefe de Bodega	Reducir quiebres de stock	Alta
Gerente de Compras	Optimizar órdenes de compra	Alta
Analista de Datos	Acceso a métricas de predicción	Media
Dirección Empresa	Reducir capital inmovilizado	Alta
Equipo de TI	Integración con sistemas existentes	Media

3.2. Usuarios

Los usuarios directos del sistema son el **Jefe de Bodega** (consulta alertas y genera órdenes) y el **Analista de Datos** (monitorea métricas de forecasting e inventario).

3.3. Roles

Los roles del sistema son: **Administrador** (gestión de productos y configuración) y **Analista** (visualización de dashboards, forecasts y órdenes).

3.4. Requisitos del Sistema

A continuación se describen los requisitos funcionales principales del sistema.

Table 3.2: Requisito RF-01: Ejecutar Predicción de Demanda por SKU

Campo	Descripción
ID	RF-01
Nombre	Ejecutar Predicción de Demanda por SKU
Descripción	El sistema debe generar una predicción de demanda para un SKU dado, seleccionando automáticamente el modelo con menor MAPE.
Actor	Analista de Datos
Prioridad	Alta

Table 3.3: Requisito RF-02: Generar Alerta de Reabastecimiento

Campo	Descripción
ID	RF-02
Nombre	Generar Alerta de Reabastecimiento
Descripción	El sistema debe detectar SKUs cuyo nivel de inventario está por debajo del punto de reorden s y generar una alerta visible en el dashboard.
Actor	Jefe de Bodega
Prioridad	Alta

Table 3.4: Requisito RF-03: Crear Orden de Compra Automática

Campo	Descripción
ID	RF-03
Nombre	Crear Orden de Compra Automática
Descripción	El sistema debe generar automáticamente una orden de compra para los SKUs en alerta, calculando la cantidad óptima según EOQ y la política (s, S) .
Actor	Jefe de Bodega
Prioridad	Alta

Table 3.5: Requisito RF-04: Visualizar Dashboard de Métricas

Campo	Descripción
ID	RF-04
Nombre	Visualizar Dashboard de Métricas
Descripción	El sistema debe mostrar gráficos de demanda histórica y predicha, MAPE por modelo y SKU, nivel de inventario y estado de órdenes.
Actor	Analista de Datos / Jefe de Bodega
Prioridad	Media

Table 3.6: Requisito RF-05: Registrar Negocio en el Sistema

Campo	Descripción
ID	RF-05
Nombre	Registrar Negocio en el Sistema
Descripción	El sistema debe permitir registrar una distribuidora o tienda con nombre, RUT y ciudad, asignándole un business_id único que aísla sus datos del resto de los negocios.
Actor	Administrador
Prioridad	Alta

Table 3.7: Requisito RF-06: Ingestar Datos de Ventas mediante IA

Campo	Descripción
ID	RF-06
Nombre	Ingestar Datos de Ventas mediante IA
Descripción	El sistema debe aceptar archivos en formato imagen (JPG, PNG), planilla Excel/CSV o PDF, extraer automáticamente los registros de ventas usando Claude (Anthropic), mostrar un preview al usuario para revisión y confirmar la carga a sales_history bajo el business_id del negocio.
Actor	Administrador / Jefe de Bodega
Prioridad	Alta

3.5. Planificación Sprint

Table 3.8: Planificación de Sprints

Sprint	Período	Objetivo	Responsable
1	Sem 1–2	Setup del entorno, ETL base, estructura repo	Cristóbal
2	Sem 3–4	Modelos ARIMA y Prophet, API /forecast (mock)	Int.1 + C
3	Sem 5–6	XGBoost, LSTM y AMS walk-forward	Int.1
4	Sem 7–8	EOQ, política (s, S), generador de órdenes	Int.2
5	Sem 9–10	API conectada a BD real, ingesta IA, multi-tenancy, dashboard	Cristóbal
6	Sem 11–12	Pruebas, métricas, retrospectiva, entrega final	Todos

4. Product Backlog y Sprint Backlog

4.1. Product Backlog

Table 4.1: Product Backlog — SmartSupply

ID	Rol	Historia de Usuario	Pts	Sprint
US-01	Analista	Quiero ejecutar una predicción de demanda por SKU para planificar compras.	8	2
US-02	Jefe Bodega	Quiero recibir alertas de quiebre de stock para actuar a tiempo.	8	4
US-03	Jefe Bodega	Quiero generar una orden de compra automática desde la alerta.	5	4
US-04	Analista	Quiero ver el dashboard con gráficos de demanda y stock.	8	5
US-05	Analista	Quiero comparar el MAPE de los modelos por SKU.	5	3
US-06	Admin	Quiero registrar nuevos productos con sus parámetros de inventario.	3	1
US-07	Admin	Quiero cargar datos históricos de ventas via ETL.	8	1
US-08	Analista	Quiero ver la evolución histórica de ventas de un SKU.	5	5
US-09	Jefe Bodega	Quiero consultar el historial de órdenes de compra generadas.	3	4
US-10	Admin	Quiero configurar los parámetros EOQ por SKU.	3	4
US-11	Analista	Quiero descargar el reporte de predicciones en formato CSV.	2	5

ID	Rol	Historia de Usuario	Pts	Sprint
US-12	Analista	Quiero ver el nivel de servicio y la tasa de quiebre por período.	5	6
US-13	Admin	Quiero simular el impacto de cambiar los parámetros (s, S) .	5	6
US-14	Admin	Quiero registrar mi negocio en el sistema para aislar mis datos.	3	5
US-15	Admin	Quiero subir una foto o Excel con mis ventas y que el sistema las cargue automáticamente.	13	5

4.2. Sprint Backlog — Sprint 1

Table 4.2: Sprint Backlog — Sprint 1: Setup y ETL Base

Tarea	Descripción	Estado	Horas
T-01	Configurar repositorio GitHub y estructura de carpetas	Completado	3
T-02	Implementar ETL: descompresión del dataset Kaggle	Completado	2
T-03	Implementar ETL: limpieza y normalización (02_clean.py)	En progreso	8
T-04	Configurar base de datos PostgreSQL en Supabase	Pendiente	4
T-05	Implementar ETL: carga a Supabase (03_load_supabase.py)	Pendiente	6
T-06	Crear estructura base del backend FastAPI	Completado	4

5. Arquitectura del Sistema

5.1. Visión General

SmartSupply sigue una arquitectura de cuatro capas:

Table 5.1: Arquitectura en Cuatro Capas

Capa	Tecnología	Responsabilidad
Presentación	React 18 + Vite + Recharts	Dashboard web, visualización, ingesta
Lógica	FastAPI 0.111 + Python	API REST, forecasting, inventario
Inteligencia	Claude API (Anthropic)	Extracción e interpretación de datos no estructurado
Datos	PostgreSQL via Supabase	Almacenamiento multi-tenant con RLS

5.2. Diagrama de Componentes (Vista de Desarrollo)

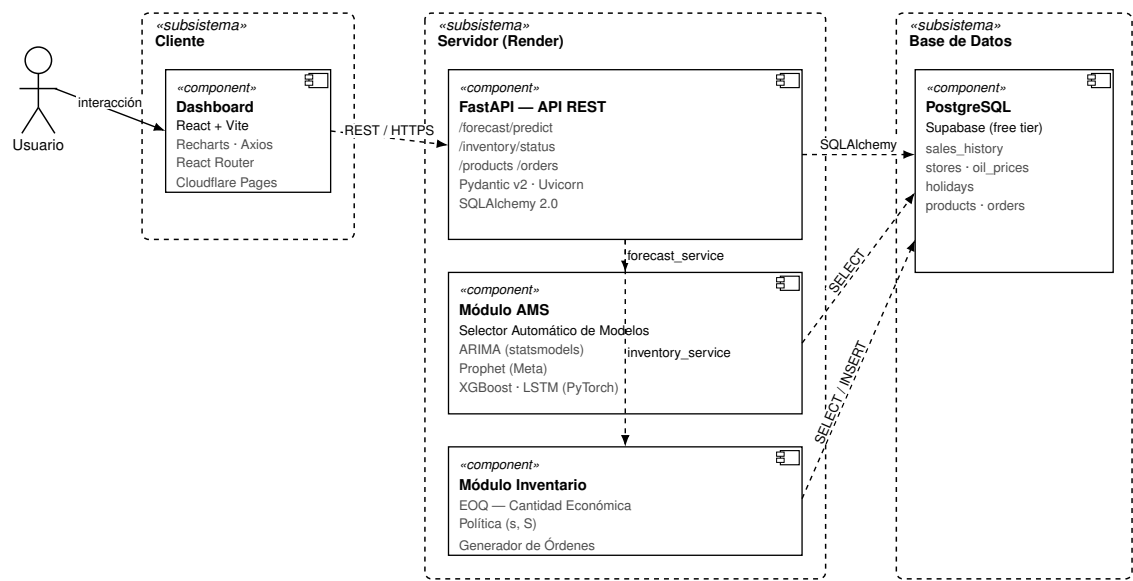


Figure 5.1: Arquitectura de componentes de SmartSupply

5.3. Diagrama de Base de Datos (Vista Lógica)

El esquema relacional de SmartSupply consta de cinco tablas. La tabla central es `businesses`, que registra cada negocio con un identificador único (`business_id`). Las tablas `sales_history` y `stores` referencian `businesses` mediante llave foránea, garantizando el aislamiento de datos entre negocios (multi-tenancy por fila). Row Level Security (RLS) de Supabase refuerza este aislamiento a nivel de base de datos. El sistema no depende de variables exógenas externas fijas: cualquier feature adicional (feriados, eventos, precios) puede ser incorporada por cada negocio a través del módulo de ingesta IA.

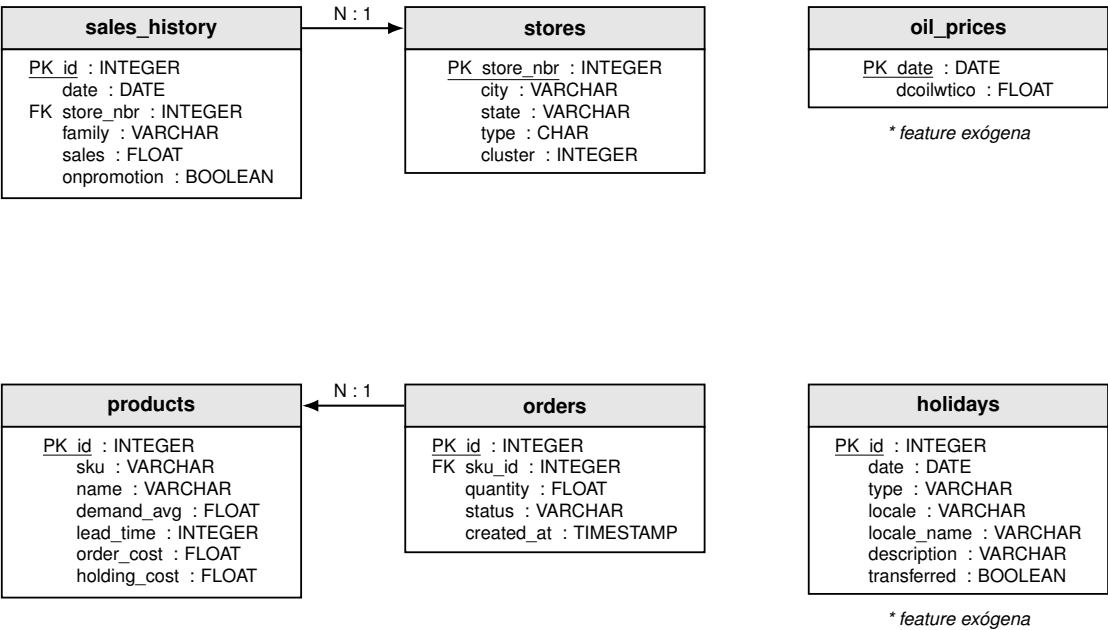


Figure 5.2: Diagrama de Base de Datos de SmartSupply

5.4. Diagrama de Secuencia — Predicción de Demanda

La Figura 5.3 muestra el flujo completo de una solicitud de predicción. El usuario selecciona un SKU en el dashboard, el cual invoca el endpoint `POST /forecast/predict`. La API consulta el historial de ventas y las *features* exógenas en PostgreSQL, luego delega al módulo AMS que entrena y evalúa los cuatro modelos en un ciclo *loop*, retornando la predicción del modelo con menor MAPE.

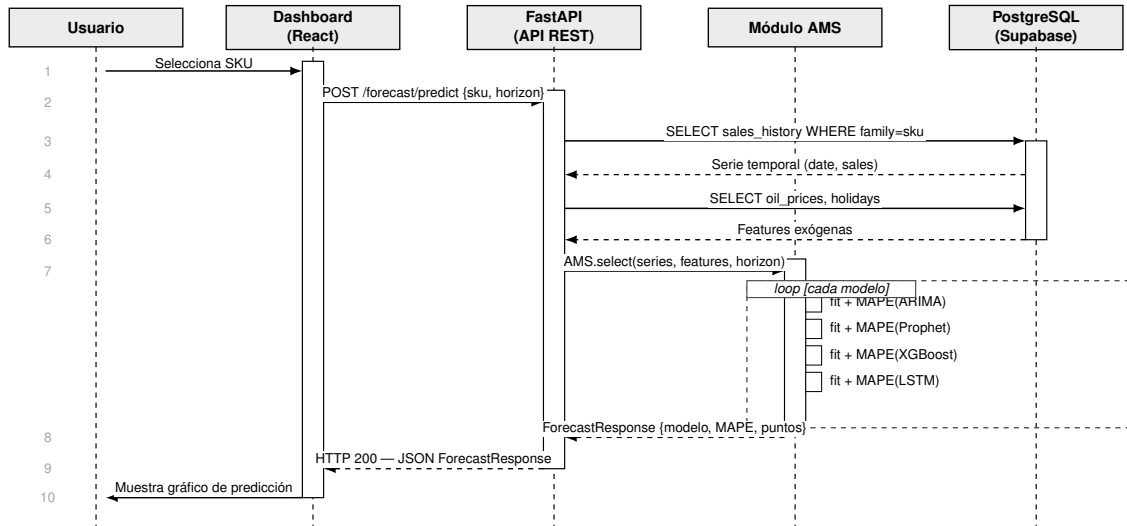


Figure 5.3: Diagrama de Secuencia — Predicción de demanda via AMS

5.5. Diagrama de Despliegue (Vista Física)

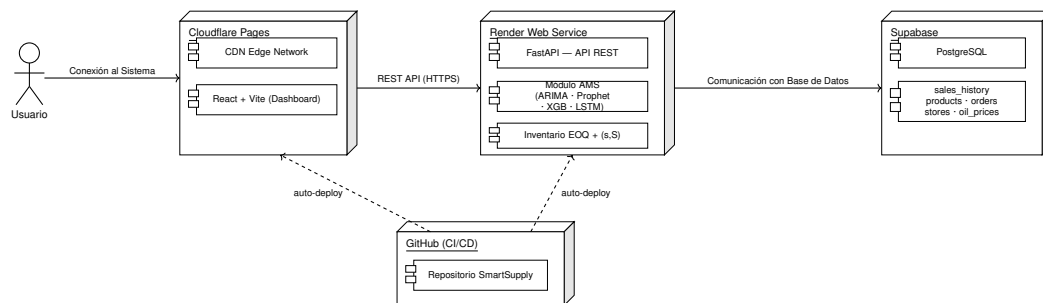


Figure 5.4: Infraestructura de despliegue de SmartSupply (Cloudflare Pages + Render + Supabase)

5.6. Módulo de Ingesta Inteligente

Una de las barreras de entrada más frecuentes para las distribuidoras chilenas es la ausencia de datos históricos en formato digital estructurado. SmartSupply resuelve este problema mediante un módulo de ingesta basado en inteligencia artificial que acepta cualquier formato de entrada.

5.6.1. Flujo de Ingesta

1. El usuario sube un archivo (imagen, Excel o PDF) via el endpoint `POST /api/ingest/preview`.
2. El backend detecta el tipo de archivo y selecciona la estrategia de extracción:

- **Imagen** (JPG, PNG, WEBP): se envía a Claude Opus 4.7 con capacidades de visión mediante la API de Anthropic.
 - **Excel/CSV**: si las columnas son reconocibles (fecha, producto, ventas), se mapean directamente con pandas. Si son ambiguas, se envía una muestra en texto a Claude Sonnet 4.6.
 - **PDF**: se envía a Claude Opus 4.7 usando el tipo de contenido `document` de la API.
3. Claude devuelve un JSON normalizado con registros del tipo `{date, family, sales, onpromotion}`.
 4. El sistema retorna un **preview** al usuario: número de registros encontrados, rango de fechas, familias detectadas y advertencias.
 5. El usuario revisa el preview y confirma la carga via POST `/api/ingest/confirm`, especificando el `business_id` y `store_nbr` destino.
 6. Los registros se insertan en `sales_history` con upsert (actualiza si ya existe, inserta si no).

5.6.2. Prompt de Extracción

El prompt enviado a Claude instruye al modelo a identificar columnas de fecha, producto y cantidad vendida, normalizar fechas al formato ISO 8601, y estandarizar los nombres de categoría en mayúsculas. El modelo devuelve **únicamente** un JSON válido, evitando texto adicional que dificulte el parseo. En caso de datos ambiguos o ilegibles, Claude los documenta en el campo `warnings` del JSON de respuesta.

6. Desarrollo

6.1. Pipeline ETL

El pipeline ETL consta de tres etapas implementadas como scripts Python:

1. `01_download_kaggle.py`: descomprime el archivo `.zip` descargado de Kaggle en `forecasting/data/`.
2. `02_clean.py`: elimina valores negativos de ventas, genera features temporales (lags, medias móviles, día de semana, mes) y produce `train_clean.csv`.
3. `03_load_supabase.py`: carga el CSV limpio a la tabla `sales_history` de PostgreSQL usando SQLAlchemy, con `ON CONFLICT DO NOTHING` para re-runs idempotentes.

6.2. Módulo de Forecasting

Cada modelo implementa la interfaz común:

```
class BaseModel:
    def fit(self, series: pd.Series) -> None: ...
    def predict(self, horizon: int) -> pd.Series: ...
```

El AMS itera sobre los cuatro modelos, calcula el MAPE en la ventana de validación y retorna el modelo de menor error junto con sus predicciones para el horizonte requerido.

6.3. Módulo de Inventario

El módulo de inventario recibe las predicciones del AMS y calcula:

- Q^* (EOQ) para determinar la cantidad óptima de pedido.
- s (punto de reorden) = demanda durante lead time + stock de seguridad.
- S (nivel objetivo) = $s + Q^*$.
- Genera una `PurchaseOrder` cuando el inventario cae bajo s .

6.4. API REST

La API expone siete routers organizados por dominio funcional. Todos los endpoints están documentados automáticamente via OpenAPI en `/docs`.

Table 6.1: Endpoints de la API REST

Método	Ruta	Descripción
POST	<code>/api/forecast/predict</code>	Predice demanda via AMS
GET	<code>/api/forecast/models</code>	Lista modelos disponibles
GET	<code>/api/inventory/alerts</code>	SKUs bajo punto de reorden
GET	<code>/api/inventory/{sku}/metrics</code>	EOQ, s , S , rotación
GET	<code>/api/products/</code>	Lista de productos
POST	<code>/api/orders/generate</code>	Genera órdenes automáticas
GET	<code>/api/sales/date-range</code>	Rango de fechas disponible del negocio
GET	<code>/api/sales/families</code>	Categorías de productos del negocio
GET	<code>/api/sales/history</code>	Historial de ventas por categoría
GET	<code>/api/sales/summary</code>	Resumen de ventas agregado por categoría
GET	<code>/api/sales/stores</code>	Tiendas de un negocio
POST	<code>/api/businesses/</code>	Registra un nuevo negocio
GET	<code>/api/businesses/</code>	Lista negocios registrados
POST	<code>/api/ingest/preview</code>	Extrae datos de archivo via Claude IA
POST	<code>/api/ingest/confirm</code>	Confirma y carga los datos a BD

6.5. Dashboard Web

El dashboard React muestra cinco vistas principales:

1. **Inicio:** KPIs (MAPE promedio, stock crítico, órdenes pendientes).
2. **Forecasting:** gráfico de demanda histórica + predicha por SKU.
3. **Inventario:** tabla de SKUs con nivel actual, s , S y estado.
4. **Órdenes:** historial y estado de órdenes de compra.
5. **Ingesta:** formulario para subir archivos (imagen, Excel, PDF), visualización del preview extraído por Claude y confirmación de carga.

7. Pruebas y Resultados

7.1. Pruebas Unitarias

Table 7.1: Prueba Unitaria PU-01: Predicción de Demanda

Campo	Detalle
ID	PU-01
Descripción	Verificar que el endpoint <code>POST /forecast/predict</code> retorna una lista de puntos de predicción con fecha y valor.
Precondición	Base de datos con datos de <code>sales_history</code> cargados.
Resultado esp.	HTTP 200, lista de <code>ForecastPoint</code> con 30 elementos.
Resultado real	[Pendiente — Sprint 5]
Estado	Pendiente

Table 7.2: Prueba Unitaria PU-02: Alerta de Reabastecimiento

Campo	Detalle
ID	PU-02
Descripción	Verificar que el endpoint <code>GET /inventory/alerts</code> retorna los SKUs cuyo inventario está bajo el punto de reorden.
Precondición	Inventario con al menos un SKU en estado crítico.
Resultado esp.	HTTP 200, lista de <code>InventoryStatus</code> con campo <code>alert=true</code> .
Resultado real	[Pendiente — Sprint 5]
Estado	Pendiente

7.2. Resultados del Módulo AMS

Los resultados cuantitativos del AMS se presentarán al finalizar el Sprint 3, una vez entrenados y evaluados los cuatro modelos sobre el dataset completo. Se reportarán: MAPE por modelo y familia de producto, distribución de modelos seleccionados por SKU, y comparación frente a un modelo único (baseline ARIMA).

8. Retrospectiva del Sprint

8.1. Comprobación de Sprint 1

Table 8.1: Recuento de Tareas — Sprint 1

Estado	Tareas	Porcentaje
Completadas	3	50 %
En progreso	1	17 %
Pendientes	2	33 %

8.2. Retrospectiva del Sprint 1

¿Qué salió bien?

- La estructura del repositorio GitHub fue completada en el primer día del Sprint.
- El backend FastAPI quedó operativo con todos los routers y schemas definidos.
- Se logró integrar exitosamente el dataset de Kaggle al pipeline ETL.

¿Qué se puede mejorar?

- La limpieza del dataset requirió más tiempo del estimado; se debe asignar más puntos de historia a las tareas de ETL en próximos sprints.
- La configuración de credenciales de Supabase generó un bloqueo de 2 días.

Acciones de mejora para Sprint 2:

1. Documentar las variables de entorno requeridas en `.env.example` antes de comenzar el Sprint.
2. Estimar las tareas de datos con un factor $\times 1,5$ respecto a la estimación inicial.

8.3. Gráfico Burn-down

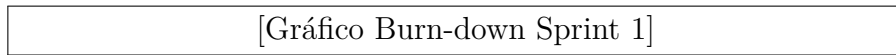


Figure 8.1: Burn-down del Sprint 1

9. Conclusiones

9.1. Conclusiones respecto a los Objetivos

9.1.1. Objetivos Específicos

Los objetivos específicos del proyecto se encuentran en diferentes estados de avance:

1. **OE-1 (Modelos de Forecasting):** los cuatro modelos han sido implementados como esqueletos con la interfaz `fit/predict`; la implementación completa se realizará en los Sprints 2 y 3.
2. **OE-2 (AMS):** la lógica de selección walk-forward está diseñada; su implementación completa está planificada para el Sprint 3.
3. **OE-3 (Políticas de Inventario):** EOQ y (s, S) están definidos matemáticamente e implementados como esqueletos; la implementación completa corresponde al Sprint 4.
4. **OE-4 (API REST): Completado.** El backend FastAPI expone siete routers (forecast, inventory, products, orders, sales, ingest, businesses), conectado a PostgreSQL/Supabase y retornando datos reales.
5. **OE-5 (Dashboard):** la estructura del proyecto React está creada; el desarrollo de componentes visuales es el objetivo del Sprint 5.
6. **OE-6 (ETL): Completado.** Los scripts `02_clean.py` y `03_load_supabase.py` están implementados; los datos de Corporación Favorita están cargados en Supabase bajo `business_id=1`.
7. **OE-7 (Ingesta IA): Completado.** El módulo `IngestService` utiliza Claude Opus 4.7 para extraer datos de imágenes, Excel y PDF. Los endpoints `/api/ingest/preview` y `/api/ingest/confirm` están operativos. La arquitectura multi-tenant con `business_id` y RLS está implementada en Supabase.

9.1.2. Objetivo General

El proyecto SmartSupply avanza conforme al plan de trabajo Scrum. La hipótesis central —que la selección automática por SKU reduce el MAPE frente a un modelo único— será validada experimentalmente en el Sprint 3 con los resultados del módulo AMS.

9.2. Conclusión respecto a la Metodología Scrum

La metodología Scrum ha permitido organizar el trabajo entre los tres integrantes con roles y responsabilidades claras, entregando incrementos funcionales al final de cada Sprint y facilitando la detección temprana de bloqueos (como el problema de credenciales de Supabase en el Sprint 1).

9.3. Proyecciones

Al concluir los 6 Sprints, SmartSupply estará en condiciones de:

- Predecir la demanda de 33 familias de productos en 54 tiendas con error cuantificado por MAPE.
- Generar automáticamente órdenes de compra calibradas por la política (s, S) .
- Ofrecer un dashboard web accesible para jefes de bodega y analistas.
- Publicar una API REST documentada (Swagger/OpenAPI) consumible por sistemas ERP.
- Permitir a distribuidoras chilenas sin sistemas digitalizados incorporar sus datos históricos mediante el módulo de ingesta IA, eliminando la barrera de entrada al sistema.
- Operar como plataforma multi-tenant, permitiendo que múltiples negocios usen la misma instancia con datos completamente aislados.

Referencias

- [1] Box, G. E. P., & Jenkins, G. M. (1976). *Time Series Analysis: Forecasting and Control*. Holden-Day.
- [2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- [3] Harris, F. W. (1913). How many parts to make at once. *Factory, The Magazine of Management*, 10(2), 135–136.
- [4] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [5] Kaggle. (2017). *Store Sales — Time Series Forecasting*. <https://www.kaggle.com/competitions/store-sales-time-series-forecasting>
- [6] Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and Machine Learning forecasting methods: Concerns and ways forward. *PLOS ONE*, 13(3), e0194889.
- [7] Ramírez, P., & Concha, C. (2021). Inventarios en empresas distribuidoras chilenas: desafíos y oportunidades. *Revista Chilena de Ingeniería Industrial*, 39(2), 45–58.
- [8] Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. <https://scrumguides.org/scrum-guide.html>
- [9] Shearer, C. (2000). The CRISP-DM model: The new blueprint for data mining. *Journal of Data Warehousing*, 5(4), 13–22.
- [10] Silver, E. A., Pyke, D. F., & Peterson, R. (1998). *Inventory Management and Production Planning and Scheduling* (3.^a ed.). Wiley.
- [11] Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45.
- [12] Tiacci, L., & Saetta, S. (2009). An approach to evaluate the impact of interaction

between demand forecasting method and stock control policy on the inventory system performances. *International Journal of Production Economics*, 118(1), 63–71.

- [13] Tabarroni, S. (2025). *Desarrollo de Sistema de Predicciones Futuras de las Acciones de Microsoft mediante Redes Neuronales*. Universidad Andrés Bello.
- [14] Anthropic. (2025). *Claude API Documentation — Vision and Document Understanding*. <https://docs.anthropic.com>
- [15] Curino, C., Jones, E., Shah, M., & Madden, S. (2010). Relational cloud: A database-as-a-service for the cloud. *CIDR Conference*, 235–240.